

LAPORAN PRAKTIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 9”

Disusun Oleh:

Haykal Muhammad Dahnil

2511532014

Dosen Pengampu :	Dr. Wahyudi, S.T, M.T.
Asisten Praktikum :	Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga laporan praktikum Praktikum Struktur Data untuk pertemuan pekan ke-9 ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk mendokumentasikan hasil praktikum mengenai dua struktur data non-linear yang sangat penting, yaitu Binary Tree dan Graph. Fokus utama pada pekan ini adalah memahami representasi simpul pohon biner, teknik kunjungan pohon (Preorder, Inorder, Postorder), serta algoritma penelusuran graf menggunakan metode Breadth-First Search (BFS) dan Depth-First Search (DFS).

Penulis mengucapkan terima kasih kepada dosen pengampu serta asisten praktikum atas bimbingan yang telah diberikan selama pelaksanaan praktikum ini. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan.

Padang, 11 Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	3
2.1 Class BubleSort_2511532014	3
2.2 Class SelectionSort_2511532014.....	5
2.3 Class InsertionSort_2511532014.....	8
2.4 Class InsertionSortGUI_2511532014.....	11
BAB III KESIMPULAN	16
DAFTAR PUSTAKA	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada pekan-pekan sebelumnya, praktikum Struktur Data berfokus pada struktur data linear seperti Array, Stack, Queue, dan Linked List di mana elemen-elemen data disusun secara berurutan. Namun, banyak permasalahan komputasi di dunia nyata yang tidak dapat direpresentasikan secara linear. Oleh karena itu, diperlukan struktur data non-linear yang mampu menggambarkan hubungan hierarkis dan jaringan antar data, yaitu Tree (Pohon) dan Graph (Graf).

Binary Tree adalah struktur data hierarkis yang terdiri dari simpul-simpul (nodes), di mana setiap simpul maksimal hanya dapat memiliki dua buah cabang anak, yang disebut sebagai Left Child (anak kiri) dan Right Child (anak kanan). Pohon biner ini jamak diaplikasikan dalam pembuatan struktur indeks basis data dan ekspresi matematika.

Di sisi lain, Graph adalah struktur data yang lebih umum, terdiri dari kumpulan simpul (Vertices/Nodes) yang dihubungkan oleh garis penunjuk (Edges). Graf sangat ideal untuk memodelkan jaringan pertemanan media sosial, peta rute perjalanan, maupun jaringan komputer. Praktikum pekan ke-9 ini bertujuan untuk memberikan pemahaman mendalam mengenai implementasi kode dan teknik penelusuran pada kedua struktur data non-linear tersebut.

1.2 Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memahami konsep dasar, terminologi, dan struktur representasi data non-linear Tree dan Graph.
2. Mampu mengimplementasikan simpul Binary Tree dan fungsi kunjungan (Preorder, Inorder, Postorder) secara rekursif menggunakan Java.
3. Mampu menghitung jumlah simpul (node counting) dan melakukan pencarian nilai (searching) pada Binary Tree.
4. Mampu memodelkan struktur graf menggunakan representasi Adjacency List memanfaatkan Map dan List.

5. Mampu mengimplementasikan dan menganalisis algoritma penelusuran graf berbasis Depth-First Search (DFS) dan Breadth-First Search (BFS).

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini meliputi:

1. Penguasaan Logika Non-Linear: Mahasiswa dapat beradaptasi dari cara berpikir linear ke hierarkis/jaringan dalam memanipulasi referensi memori objek objek komputasi.
2. Keterampilan Algoritma Rekursif: Mengasah ketajaman logika mahasiswa dalam merancang fungsi rekursif yang bersih dan efisien untuk menelusuri struktur pohon.
3. Kemampuan Pemodelan Masalah: Mahasiswa mampu mentransformasikan studi kasus jaringan (seperti penelusuran rute) ke dalam kode pemrograman graf yang aplikatif.

BAB II PEMBAHASAN

2.1 Class Node_2511532014

2.1.1 Langkah Pengerjaan

1. Membuat kelas bernama Node_2511532014 di dalam package pekan9_2511532014.
2. Mendeklarasikan atribut data integer data_2014 serta dua variabel referensi objek bertipe sama: left_2014 dan right_2014.
3. Menyusun konstruktor untuk menginisialisasi nilai data simpul dan mengatur pointer left_2014 serta right_2014 ke nilai null.
4. Membuat metode Setter (setLeft_2014, setRight_2014) dengan validasi pengaman; simpul anak hanya dipasang jika variabel penunjuk saat ini masih bernilai null.
5. Membuat metode Getter (getLeft_2014, getRight_2014, getData_2014) dan fungsi pengubah data (setData_2014).
6. Mengimplementasikan fungsi kunjungan Preorder (printPreorder_2014) secara rekursif: cetak data simpul saat ini, panggil rekursif ke anak kiri, lalu panggil rekursif ke anak kanan.
7. Mengimplementasikan fungsi kunjungan Postorder (printPostorder_2014) secara rekursif: panggil rekursif ke anak kiri, panggil rekursif ke anak kanan, baru kemudian cetak data simpul saat ini.
8. Mengimplementasikan fungsi kunjungan Inorder (printInorder_2014) secara rekursif: panggil rekursif ke anak kiri, cetak data simpul saat ini, baru kemudian panggil rekursif ke anak kanan.

2.1.2 Kode Program

```
1 package pekan9_2511532014;
2
3 public class Node_2511532014 {
4     int data_2014;
5     Node_2511532014 left_2014;
6     Node_2511532014 right_2014;
7
8     public Node_2511532014(int data_2014) {
9         this.data_2014 = data_2014;
10        left_2014 = null;
11        right_2014 = null;
12    }
13
14    public void setLeft_2014(Node_2511532014 node) {
15        if (left_2014 == null)
16            left_2014 = node;
17    }
18
19    public void setRight_2014(Node_2511532014 node) {
20        if (right_2014 == null)
21            right_2014 = node;
22    }
23 }
```

```

24     public Node_2511532014 getLeft_2014() {
25         return left_2014;
26     }
27
28     public Node_2511532014 getRight_2014() {
29         return right_2014;
30     }
31
32     public int getData_2014() {
33         return data_2014;
34     }
35
36     public void setData_2014(int data_2014) {
37         this.data_2014 = data_2014;
38     }
39
40     // Traversal Methods
41     void printPreorder_2014(Node_2511532014 node_2014) {
42         if (node_2014 == null)
43             return;
44         System.out.print(node_2014.data_2014 + " ");
45         printPreorder_2014(node_2014.left_2014);
46         printPreorder_2014(node_2014.right_2014);
47     }
48
49     void printPostorder_2014(Node_2511532014 node_2014) {
50         if (node_2014 == null)
51             return;
52         printPostorder_2014(node_2014.left_2014);
53         printPostorder_2014(node_2014.right_2014);
54         System.out.print(node_2014.data_2014 + " ");
55     }
56
57     void printInorder_2014(Node_2511532014 node_2014) {
58         if (node_2014 == null)
59             return;
60         printInorder_2014(node_2014.left_2014);
61         System.out.print(node_2014.data_2014 + " ");
62         printInorder_2014(node_2014.right_2014);
63     }
64
65     public String print_2014() {
66         return this.print_2014("", true, "");
67     }
68
69     public String print_2014(String prefix_2014, boolean
isTail_2014, String sb_2014) {
70         if (right_2014 != null) {
71             right_2014.print_2014(prefix_2014 + (isTail_2014 ? "|"
72 " : "      "), false, sb_2014);
73         }
74         System.out.println(prefix_2014 + (isTail_2014 ? "\\--" :
"/--") + data_2014);
75         if (left_2014 != null) {
76             left_2014.print_2014(prefix_2014 + (isTail_2014 ? "
" : "|      "), true, sb_2014);
77         }

```

```
77         return sb_2014;
78     }
79 }
80
```

2.1.3 Analisis

Kelas `Node_2511532014` mendefinisikan blok bangunan paling mendasar dari struktur Binary Tree di memori heap komputer. Karakteristik utama yang membedakannya dari simpul Single Linked List adalah keberadaan dua buah pointer referensi (`left_2014` dan `right_2014`). Perbedaan ini menciptakan struktur percabangan hierarkis ke bawah.

Fungsi penelusuran (traversal) yang diimplementasikan di kelas ini memanfaatkan keunggulan teknik Rekursi. Rekursi sangat ideal untuk Tree karena struktur pohon pada prinsipnya adalah kumpulan dari sub-pohon (sub-trees) yang lebih kecil. Kondisi `if (node_2014 == null) return;` bertindak sebagai base case (kasus dasar) yang krusial untuk menghentikan pemanggilan fungsi agar program tidak mengalami error akibat tumpukan memori penuh (`StackOverflowError`). Perbedaan urutan penempatan baris perintah pencetakan data (`System.out.print`) di antara panggilan rekursif anak kiri dan kanan menentukan karakteristik hasil penelusuran data secara radikal, melahirkan tiga pola pembacaan pohon yang unik.

2.2 Class `BTree_2511532014`

2.2.1 Langkah Pengerjaan

1. Membuat kelas `BTree_2511532014` sebagai pembungkus utama manajemen pohon biner.
2. Mendeklarasikan atribut `root_2014` bertipe objek simpul untuk memegang titik puncak teratas pohon.
3. Menyusun konstruktor default untuk mengeset `root_2014 = null` saat pohon pertama kali dibentuk.
4. Membuat metode pencarian publik `search_2014(int data_2014)`. Metode ini meneruskan pencarian ke fungsi privat internal secara rekursif.
5. Logika fungsi pencarian privat memeriksa apakah simpul saat ini cocok dengan nilai target. Jika tidak, telusuri cabang anak kiri secara rekursif, dan jika belum ditemukan, lanjutkan telusuri cabang anak kanan secara rekursif.

6. Menyusun metode penjembaran publik untuk mencetak isi pohon secara Inorder, Preorder, dan Postorder dengan memanggil fungsi internal milik kelas Node_2511532014.
7. Mengimplementasikan metode penghitung simpul privat countNodes_2014(Node_2511532014 node_2014). Fungsi ini menjumlahkan secara rekursif: 1 + countNodes(anak_kiri) + countNodes(anak_kanan).

2.2.2 Kode Program

```

1 package pekan9_2511532014;
2
3 public class BTree_2511532014 {
4     private Node_2511532014 root_2014;
5     private Node_2511532014 currentNode_2014;
6
7     public BTree_2511532014() {
8         root_2014 = null;
9     }
10
11     public boolean search_2014(int data_2014) {
12         return search_2014(root_2014, data_2014);
13     }
14
15     private boolean search_2014(Node_2511532014 node_2014, int
data_2014) {
16         if (node_2014 == null)
17             return false;
18         if (node_2014.getData_2014() == data_2014)
19             return true;
20         if (node_2014.getLeft_2014() != null)
21             if (search_2014(node_2014.getLeft_2014(),
data_2014))
22                 return true;
23         if (node_2014.getRight_2014() != null)
24             if (search_2014(node_2014.getRight_2014(),
data_2014))
25                 return true;
26         return false;
27     }
28
29     public void printInorder_2014() {
30         root_2014.printInorder_2014(root_2014);
31     }
32
33     public void printPreOrder_2014() {
34         root_2014.printPreorder_2014(root_2014);
35     }
36
37     public void printPostOrder_2014() {
38         root_2014.printPostorder_2014(root_2014);
39     }
40
41     public Node_2511532014 getRoot_2014() {
42         return root_2014;
43     }

```

```

44
45     public boolean isEmpty_2014() {
46         return root_2014 == null;
47     }
48
49     public int countNodes_2014() {
50         return countNodes_2014(root_2014);
51     }
52
53     private int countNodes_2014(Node_2511532014 node_2014) {
54         int count_2014 = 1;
55         if (node_2014 == null) {
56             return 0;
57         } else {
58             count_2014 +=
59 countNodes_2014(node_2014.getLeft_2014());
60             count_2014 +=
61 countNodes_2014(node_2014.getRight_2014());
62             return count_2014;
63         }
64     }
65
66     public void print_2014() {
67         root_2014.print_2014();
68     }
69
70     public Node_2511532014 getCurrent_2014() {
71         return currentNode_2014;
72     }
73
74     public void setCurrent_2014(Node_2511532014 node_2014) {
75         this.currentNode_2014 = node_2014;
76     }
77
78     public void setRoot_2014(Node_2511532014 root_2014) {
79         this.root_2014 = root_2014;
80     }
81 }

```

2.2.3 Analisis

Kelas BTree_2511532014 mengadopsi prinsip desain fasad (Facade Pattern) untuk menyembunyikan kompleksitas operasi internal pohon dari pengguna luar. Analisis pada fungsi countNodes_2014 memperlihatkan logika akumulasi rekursif yang elegan. Jika sebuah simpul tidak bernilai kosong, program menginisialisasi nilai counter dasar sebesar 1 (menghitung simpul itu sendiri), kemudian secara rekursif menuruni cabang kiri dan cabang kanan untuk mengumpulkan jumlah simpul anak, lalu menjumlahkan seluruhnya.

Karena ini adalah pohon biner biasa (bukan Binary Search Tree / BST), data di dalamnya tidak memiliki aturan urutan penempatan nilai yang teratur. Akibat ketidakpastian posisi ini, metode pencarian data `search_2014` terpaksa mengadopsi kompleksitas waktu linier $O(n)$, di mana pada skenario terburuk program harus menyalin dan memeriksa seluruh simpul satu per satu tanpa terkecuali untuk memastikan keberadaan nilai target.

2.3 Class BtreeDriver_2511532014

2.3.1 Langkah Pengerjaan

1. Membuat kelas driver `BtreeDriver_2511532014` untuk mengeksekusi pengujian objek pohon.
2. Instansiasi objek `tree_2014` dari kelas `BTree_2511532014`, lalu cetak jumlah simpul awalnya (harus bernilai 0).
3. Membuat objek simpul utama `root_2014` dengan muatan data angka 1, lalu pasang sebagai titik puncak pohon menggunakan metode `.setRoot_2014()`.
4. Instansiasi sembilan objek simpul pendukung lainnya (`node2` hingga `node9`) dengan nilai data bervariasi.
5. Menyusun struktur hierarki pohon secara manual dengan menghubungkan simpul satu sama lain memanfaatkan fungsi `.setLeft_2014()` dan `.setRight_2014()`.
6. Memanggil metode cetak untuk memvisualisasikan hasil kunjungan pohon secara Preorder, Inorder, dan Postorder.
7. Melakukan pengujian fungsi pencarian data untuk mendeteksi keberadaan angka 5 dan angka 20 di dalam sistem pohon tersebut.

2.3.2 Kode Program

```
1 package pekan9_2511532014;
2
3 public class BtreeDriver_2511532014 {
4     public static void main(String[] args) {
5         // Membuat Pohon
6         BTree_2511532014 tree_2014 = new BTree_2511532014();
7         System.out.print("Jumlah Simpul awal pohon: ");
8         System.out.println(tree_2014.countNodes_2014());
9         // Menambahkan simpul data 1
10        Node_2511532014 root_2014 = new Node_2511532014(1);
11        // Menjadikan simpul 1 sebagai root
12        tree_2014.setRoot_2014(root_2014);
13        System.out.println("Jumlah simpul jika hanya ada root");
14        System.out.println(tree_2014.countNodes_2014());
15
16        Node_2511532014 node2_2014 = new Node_2511532014(2);
17        Node_2511532014 node3_2014 = new Node_2511532014(3);
18        Node_2511532014 node4_2014 = new Node_2511532014(4);
19        Node_2511532014 node5_2014 = new Node_2511532014(5);
```

```

20     Node_2511532014 node6_2014 = new Node_2511532014(6);
21     Node_2511532014 node7_2014 = new Node_2511532014(7);
22     Node_2511532014 node8_2014 = new Node_2511532014(8);
23     Node_2511532014 node9_2014 = new Node_2511532014(9);
24
25     root_2014.setLeft_2014(node2_2014);
26     node2_2014.setLeft_2014(node4_2014);
27     node2_2014.setRight_2014(node5_2014);
28     node4_2014.setRight_2014(node8_2014);
29     root_2014.setRight_2014(node3_2014);
30     node3_2014.setLeft_2014(node6_2014);
31     node3_2014.setRight_2014(node7_2014);
32     node6_2014.setLeft_2014(node9_2014);
33
34     //set root
35     tree_2014.setCurrent_2014(tree_2014.getRoot_2014());
36     System.out.println("Menampilkan simpul terakhir");
37
38     System.out.println(tree_2014.getCurrent_2014().getData_2014());
39     System.out.println("Jumlah simpul setelah simpul 7
ditambahkan");
40     System.out.println(tree_2014.countNodes_2014());
41     System.out.println("InOrder:");
42     tree_2014.printInorder_2014();
43     System.out.println("\nPreorder:");
44     tree_2014.printPreOrder_2014();
45     System.out.println("\nPostorder:");
46     tree_2014.printPostOrder_2014();
47     System.out.println("\nMenemapiikan simpul dalam bentuk
pohon");
48     tree_2014.print_2014();
49 }
50

```

2.3.3 Output

```

Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
Menampilkan simpul terakhir
1
Jumlah simpul setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder:
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|
| /--7
| /--3
| | \--6
| | \--9
|--1
| /--5
|--2
| /--8
|--4

```

2.3.4 Analisis

Berbeda dengan Merge Sort yang membagi array secara merata tanpa melihat nilai elemennya Quick Sort membagi sub-array berdasarkan nilai pembatas atau elemen tumpuan yang disebut Pivot. Masalah terbesar pada Quick Sort standar adalah jika data masukan sudah terurut dan kita selalu memilih elemen terakhir sebagai pivot maka pembagian sub-array menjadi sangat tidak seimbang ($O(n^2)$). Untuk mengatasi hal tersebut kode program di atas menerapkan teknik optimasi Median-of-Three pada fungsi `medianOfThree\_2014`.

Hasil eksekusi program driver memvalidasi secara empiris kebenaran teori urutan kunjungan pohon (Binary Tree Traversal). Berdasarkan kode penyusunan simpul, pohon yang terbentuk memiliki struktur hierarki yang seimbang.

- Pada kunjungan Preorder (Root-Left-Right), data dicetak langsung saat simpul pertama kali dikunjungi, menghasilkan pola 1 2 4 8... yang mencerminkan penelusuran mendalam ke ujung kiri terlebih dahulu.

- Pada kunjungan Inorder (Left-Root-Right), data dicetak setelah cabang anak kiri selesai diproses sepenuhnya. Alhasil, angka 8 (daun paling kiri bawah) dicetak sebagai elemen pembuka antrian teks.
- Pada kunjungan Postorder (Left-Right-Root), simpul puncak utama (root bernilai 1) baru dicetak di paling akhir setelah seluruh sub-pohon anak kiri dan kanan selesai dibersihkan.

Metode hitung simpul berhasil melacak penambahan kapasitas objek dari 0, 1, hingga 9 simpul secara akurat, membuktikan akurasi fungsi rekursif tumpukan. Hasil pencarian angka 5 bernilai true karena jalannya fungsi mendeteksi kecocokan data pada simpul anak kanan dari node2_2014.

2.4 Class GraphTraversal_2511532014

2.4.1 Langkah Pengerjaan

1. Membuat kelas bernama GraphTraversal_2511532014 di dalam package praktikum.
2. Mendeklarasikan objek tabel hash Map<String, List<String>> graph_2014 untuk merepresentasikan struktur Adjacency List (Daftar Ketetanggaan graf).
3. Mengimplementasikan metode addEdge_2014(String node1, String node2). Gunakan fungsi putIfAbsent untuk menginisialisasi list kosong jika simpul belum terdaftar, lalu masukkan keterhubungan secara timbal balik (karena graf tidak berarah).
4. Membuat metode printGraph_2014() untuk mendeserialisasi isi Map ke layar komputer dalam format visual hubungan ketetanggaan.
5. Mengimplementasikan metode penelusuran DFS (dfs_2014). Siapkan koleksi data unik Set<String> visited untuk mencatat simpul yang telah dikunjungi, kemudian panggil fungsi pembantu rekursif dfsHelper_2014.
6. Logika fungsi dfsHelper_2014 menandai simpul saat ini sebagai dikunjungi, mencetaknya ke layar, lalu melakukan iterasi masuk secara mendalam ke tetangga pertama yang belum pernah dikunjungi secara rekursif.
7. Mengimplementasikan metode penelusuran BFS (bfs_2014) secara iteratif. Siapkan koleksi data Set untuk penanda kunjungan dan sebuah struktur data Queue (LinkedList) sebagai antrian penampung urutan pemrosesan simpul tingkat demi tingkat.
8. Logika BFS mengambil simpul terdepan dari Queue lewat fungsi .poll(), mencetaknya, lalu memasukkan seluruh tetangga simpul tersebut yang belum dikunjungi ke dalam antrian belakang Queue.

9. Di dalam fungsi main, bangun model graf pertemanan/rute fiktif yang menghubungkan huruf A hingga F, cetak struktur ketanggaannya, lalu eksekusi metode penelusuran DFS dan BFS.

2.4.2 Kode Program

```
1 package pekan9_2511532014;
2
3 import java.util.*;
4
5 public class GraphTraversal_2511532014 {
6     private Map<String, List<String>> graph_2014 = new HashMap<>();
7
8     // Menambahkan edge (graf tak berarah)
9     public void addEdge_2014(String node1_2014, String node2_2014)
10    {
11        graph_2014.putIfAbsent(node1_2014, new ArrayList<>());
12        graph_2014.putIfAbsent(node2_2014, new ArrayList<>());
13        graph_2014.get(node1_2014).add(node2_2014);
14        graph_2014.get(node2_2014).add(node1_2014);
15    }
16
17    // Menampilkan graf awal
18    public void printGraph_2014() {
19        System.out.println("Graf Awal (Adjacency List): ");
20        for (String node_2014 : graph_2014.keySet()) {
21            System.out.print(node_2014 + " -> ");
22            List<String> neighbors_2014 =
23            graph_2014.get(node_2014);
24            System.out.println(String.join(", ",
25            neighbors_2014));
26        }
27        System.out.println();
28    }
29
30    // DFS rekursif
31    public void dfs_2014(String start_2014) {
32        Set<String> visited_2014 = new HashSet<>();
33        System.out.println("Penelusuran DFS: ");
34        dfsHelper_2014(start_2014, visited_2014);
35        System.out.println();
36    }
37
38    private void dfsHelper_2014(String current_2014, Set<String>
39    visited_2014) {
40        if (visited_2014.contains(current_2014)) return;
41        visited_2014.add(current_2014);
42        System.out.print(current_2014 + " ");
43        for (String neighbor_2014 :
44        graph_2014.getDefault(current_2014, new ArrayList<>())) {
45            dfsHelper_2014(neighbor_2014, visited_2014);
46        }
47    }
48
49    // BFS iteratif
50    public void bfs_2014(String start_2014) {
51        Set<String> visited_2014 = new HashSet<>();
```

```

47     Queue<String> queue_2014 = new LinkedList<>();
48     queue_2014.add(start_2014);
49     visited_2014.add(start_2014);
50     System.out.println("Penelusuran BFS:");
51     while (!queue_2014.isEmpty()) {
52         String current_2014 = queue_2014.poll();
53         System.out.print(current_2014 + " ");
54         for (String neighbor_2014 :
graph_2014.getDefault(current_2014, new ArrayList<>())) {
55             if (!visited_2014.contains(neighbor_2014)) {
56                 queue_2014.add(neighbor_2014);
57                 visited_2014.add(neighbor_2014);
58             }
59         }
60     }
61     System.out.println();
62 }
63
64 // Main
65 public static void main(String[] args) {
66     GraphTraversal_2511532014 graph_2014 = new
GraphTraversal_2511532014();
67
68     // Contoh graf: A-B, A-C, B-D, B-E
69     graph_2014.addEdge_2014("A", "B");
70     graph_2014.addEdge_2014("A", "C");
71     graph_2014.addEdge_2014("B", "D");
72     graph_2014.addEdge_2014("B", "E");
73
74     // Cetak graf awal
75     System.out.println("Graf Awal adalah: ");
76     graph_2014.printGraph_2014();
77
78     // Lakukan penelusuran
79     graph_2014.dfs_2014("A");
80     graph_2014.bfs_2014("A");
81 }
82 }
83

```

2.4.3 Output


```
Graf Awal adalah:  
Graf Awal (Adjacency List):  
A -> B, C  
B -> A, D, E  
C -> A  
D -> B  
E -> B  
  
Penelusuran DFS:  
A B D E C  
Penelusuran BFS:  
A B C D E
```

2.4.4 Analisis

Program ini mendemonstrasikan pemodelan struktur data Graf menggunakan struktur Adjacency List yang dinamis dan efisien memori memanfaatkan struktur data objek Map. Penggunaan struktur data penanda unik `Set<String> visited_2014` sangat krusial; karena berbeda dengan Tree yang bebas dari siklus perulangan, Graf memiliki risiko hubungan antar simpul yang berputar melingkar (cyclic). Tanpa adanya pengecekan kondisi `contains()`, program akan terjebak ke dalam lingkaran setan penelusuran tak terbatas (infinite loop).

Analisis mendalam pada hasil output memperlihatkan perbedaan karakteristik yang radikal antara algoritma DFS dan BFS:

- DFS (Depth-First Search) mengadopsi filosofi pencarian mendalam. Struktur ini memanfaatkan mekanisme tumpukan memori (Call Stack dari fungsi rekursif). Saat berada di simpul A, ia langsung melompat ke tetangga pertamanya yaitu B, lalu menuruni relasi B ke D. Setelah mencapai ujung buntu di D, program melakukan proses lacak balik (backtracking) ke posisi B untuk mengeksplorasi cabang E, baru kemudian kembali ke awal untuk memproses rumpun C. Pola ini sangat hemat memori untuk graf yang memiliki cabang yang sangat dalam.
- BFS (Breadth-First Search) mengadopsi filosofi pencarian melebar setingkat demi setingkat (Level-order). Algoritma ini berjalan secara iteratif

memanfaatkan bantuan struktur data Queue (Antrian FIFO). Ketika simpul A diproses, seluruh tetangga langsung dari A (yaitu B dan C) dipaksa masuk ke dalam antrian memori secara bersamaan. Alhasil, program menyelesaikan kunjungan ke seluruh simpul tetangga terdekat terlebih dahulu sebelum melangkah masuk mengeksplorasi simpul yang letaknya jauh (D, E, F). Sifat penelusuran bertingkat ini menjadikan BFS sebagai fondasi utama yang paling ideal untuk memecahkan masalah pencarian rute terpendek (shortest path) pada sebuah graf jaringan komputer atau peta transportasi digital.

BAB III

KESIMPULAN

Berdasarkan praktikum pekan 9, dapat disimpulkan bahwa:

1. Representasi Struktur Hierarkis: Struktur data Binary Tree terbukti sangat efektif untuk menggambarkan relasi data hierarkis bercabang dua, di mana teknik fungsi rekursif menjadi instrumen paling ideal untuk mengelola penelusuran simpul secara elegan dan ringkas.
2. Karakteristik Tiga Pola Kunjungan: Pola kunjungan Preorder, Inorder, dan Postorder menghasilkan karakteristik urutan pembacaan data pohon yang berbeda secara fundamental. Hal ini ditentukan oleh penempatan baris instruksi eksekusi data terhadap pemanggilan rantai fungsi anak cabang kiri dan kanan.
3. Penyelesaian Masalah Siklus Graf: Pemodelan graf tak berarah menggunakan struktur Adjacency List berbasis Map terbukti sangat hemat penggunaan ruang memori komputer. Perlindungan menggunakan koleksi data unik Set wajib disertakan untuk mencegah kegagalan program akibat jebakan jalur melingkar (cyclic paths).
4. Divergensi Logika DFS vs BFS: Algoritma DFS (berbasis tumpukan rekursi) mengeksplorasi jalur graf secara menghujam ke dalam hingga mencapai ujung buntu, sedangkan algoritma BFS (berbasis antrian Queue) bergerak menyapu secara horizontal setingkat demi setingkat. Hal ini membuat BFS unggul untuk kalkulasi rute terpendek.

DAFTAR PUSTAKA